

COLLEGE OF ENGINEERING AND INFORMATION TECHNOLOGY

LABORATORY MANUAL ACTIVITY

Designing the Web-System Architecture of Your Own Project

Web-System Architectures and End-to-End Data Flow

Course	IT 110 – Web Systems and Technologies
Activity	Laboratory Activity No. 2
Output	System-specific architecture diagram, request-to-response trace, and group presentation
Maximum Score	80 points

Student / Project Information

Block: A1 - IBPH3	Date: Date: September 9, 2026
Group No.: _____	Instructor: Jean Y. Villasin, MIT
System Title: RoadWatch: Pothole and Road Damage Detection Using Mobile Dashcam Imagery with a Web-Based Monitoring System	
Members: RJ Cyrill Jirc L. Monzon Mae D. Pelipas Katrice B. Anunciado	

1. Activity Alignment and Purpose

Topic Learning Outcome: Explain common web-system architectures and trace the end-to-end flow of data through major system components.

Course Outcome Alignment: CO1 – Explain web architecture and the roles of client-side, server-side, framework, API, database, and deployment components in the end-to-end flow of a web system.

Syllabus Assessment: Web-system architecture diagram and request-to-response tracing exercise.

Purpose of the Activity

Before writing major code, your team must create an architecture blueprint that explains how the users, Flutter interface, application or API layer, database, external services, security controls, and deployment environment work together. The diagram will serve as a technical guide throughout the semester project.

Important distinction: Flutter is the client-side application framework. It is one part of the complete web system; your architecture must also show how data is processed, stored, secured, and returned to the user.

2. Intended Learning Objectives

- Identify the users, components, layers, services, and deployment resources required by the team's proposed system.
- Select and justify an appropriate architecture such as two-tier, three-tier, monolithic, service-oriented, or a defensible hybrid architecture.
- Create a clear system-specific architecture diagram using accurate component names, directional arrows, protocols, and data labels.
- Trace one important workflow from the user action through the Flutter client, backend/API, business logic, data store, and response shown to the user.

3. Activity Specifications

Mode	Existing semester-project group; all members must participate and be able to explain the diagram.
Suggested Time	One laboratory session, approximately 3 hours, plus the scheduled group presentation.
Primary Output	One system architecture diagram based on the team's actual approved project.
Supporting Output	One request-to-response trace, a 100–150 word architecture justification, and a group presentation.
Recommended Tools	diagrams.net/Draw.io, Lucidchart, Figma, Canva, PowerPoint, or an equivalent diagramming tool.
Presentation Format	5–7 minutes per group followed by 2–3 minutes of questions. Every member must explain an assigned architecture component or data-flow step.

4. Pre-Laboratory Requirements

- Approved or proposed system title and a brief statement of the community or organizational problem being addressed.
- List of primary users or roles and at least three major features of the system.
- Tentative technology stack, including Flutter, backend/API or backend-as-a-service, database, external services, and deployment platform.
- A copy of the lecture notes on web-system layers, two-tier and three-tier architectures, monolithic and service-oriented architectures, and request-response flow.

5. Part A - System Component Inventory

Complete the worksheet before drawing. Use the exact names planned for your own project. Do not write only generic labels such as “front end” or “database” when a more specific component name is already known.

Architecture Item	Your System	Notes / Examples of Evidence
System title and purpose	RoadWatch: Pothole and Road Damage Detection Using Mobile Dashcam Imagery with a Web-Based Monitoring System. It helps drivers identify and record potholes and lets dashboard users review their locations and detection statistics.	Problem solved and intended community or organization
Actors / user roles	Drivers use the mobile camera, detection, alerts and history. Dashboard users review recorded potholes and statistics.	Administrator, staff, customer, citizen, farmer, student, etc.
Flutter client	Flutter/Dart Android app with camera/live detection, image analysis, GPS, alerts, history and saving. A separate HTML/CSS/JavaScript dashboard uses Leaflet.	Flutter Web, mobile app, screens, widgets, navigation
State and client logic	The ONNX service runs detection. OfflineDetectionService keeps pending records in SharedPreferences. ApiService.createDetection uploads records and manages synchronization feedback.	Provider, Riverpod, BLoC, controller, service, repository, or equivalent
Backend / API / BaaS	FastAPI REST API with SQLAlchemy data access. Routes include POST /detections, GET /detections and GET /detections/statistics.	REST API, Supabase, Firebase, custom backend, serverless function
Business logic	The phone runs YOLO/ONNX inference and accepts only Pothole results. Saving attaches confidence, coordinates and time. Synchronization uses a client detection ID to prevent duplicate records.	Validation, rules, calculations, recommendation, approval workflow
Database and storage	PostgreSQL database roadwatch_db stores detection records. SharedPreferences stores pending mobile records. The ONNX model is bundled in mobile assets.	PostgreSQL, tables/entities, file or image storage
External services	The phone uses device GPS for location. Leaflet provides the dashboard map interface. The detection workflow uses an on-device model.	Maps/GIS, machine learning, email/SMS, payment, weather, other APIs
Security controls	Current design sends database operations through the API. Proposed controls are HTTPS, login/token and role checks, server-side field validation, restricted database access and protection of location data.	Authentication, authorization, role checks, tokens, validation

Deployment: The current development setup uses a Flutter phone and a FastAPI server on the same Wi-Fi/LAN. The recorded API address is <http://192.168.1.4:8000>. PostgreSQL and the browser dashboard are part of the local development environment.

6. Part B - Select the Architecture Style

Select the architecture that best fits your system. More than one may apply, but the final choice must be justified based on requirements—not merely because it is familiar.

Architecture	Core Structure	When It May Fit	Select
Two-tier	Client communicates directly with a data service or tightly coupled server.	Small systems with controlled scope; security and maintainability must still be addressed.	☐
Three-tier	Flutter presentation → application/API → database.	Recommended for systems that need clear separation of responsibilities and secure data access.	☒
Monolithic	Application modules are packaged and deployed as one backend application.	Small to medium systems requiring simpler deployment and coordinated modules.	☐
Service-oriented / Hybrid	Multiple services, BaaS modules, or external APIs cooperate.	Systems that genuinely need independent services such as GIS, ML, notifications, or payments.	☐

Selected architecture: Three-tier client-server architecture with on-device inference and offline synchronization.

Initial justification (3-5 sentences)

RoadWatch uses a three-tier architecture because the mobile and web interfaces, the FastAPI application layer, and the PostgreSQL data layer have different responsibilities. The Flutter phone runs the ONNX model locally, so pothole detection and alerts remain available when the server cannot be reached. FastAPI receives detection records and supplies the records and statistics used by the clients, while PostgreSQL provides persistent storage. This separation lets us update the dashboard, API or database without changing every part of the system, and it gives us a clear place to add authentication and server-side validation. The offline queue and client detection ID support later synchronization while reducing duplicate uploads.

How the selected architecture applies to RoadWatch

The presentation tier includes both the Flutter Android app and the HTML/CSS/JavaScript dashboard. The application tier is the FastAPI REST API. The data tier is PostgreSQL. On-device inference and the local pending queue extend the mobile client so it can keep useful functions available during a connection failure.

The current deployment is a local prototype. Public deployment would require a reachable hosted API, HTTPS and access controls. These additions are included as proposed design work rather than as completed features.

7. Part C - Construct the Web-System Architecture Diagram

Create one clean, readable diagram that shows the complete architecture of your own system. The diagram must be understandable even to a reviewer who has not seen your code.

No.	Required Element	What Must Be Shown
1	System identity	Place the official system title and selected architecture style at the top.
2	Actors	Show every major user role or external actor that sends requests to the system.
3	Client / presentation layer	Show Flutter Web or Flutter mobile, key screens or UI modules, routing, state management, and form validation as appropriate.
4	Communication layer	Show how components communicate: HTTPS, REST, WebSocket, SDK, JSON, or another relevant mechanism.
5	Application / API layer	Show request handlers, authentication, authorization, validation, service/business logic, and major server modules.
6	Data layer	Show the selected database, important tables/entities or data domains, file storage, relationships, and constraints at an appropriate level.
7	External services	Show only services actually used by the project, such as GIS/maps, ML model or API, email/SMS, payment, weather, or cloud storage.
8	Deployment	Show where the Flutter client, backend/API, database, and storage will run or be hosted.
9	Data flow	Use directional arrows and label important requests, responses, protocols, data formats, and returned results.
10	Security	Show where login, role checks, tokens, input validation, access control, and protected resources are handled.
11	Legend	Use consistent symbols, colors, line styles, and a brief legend.

Minimum Diagram Rule

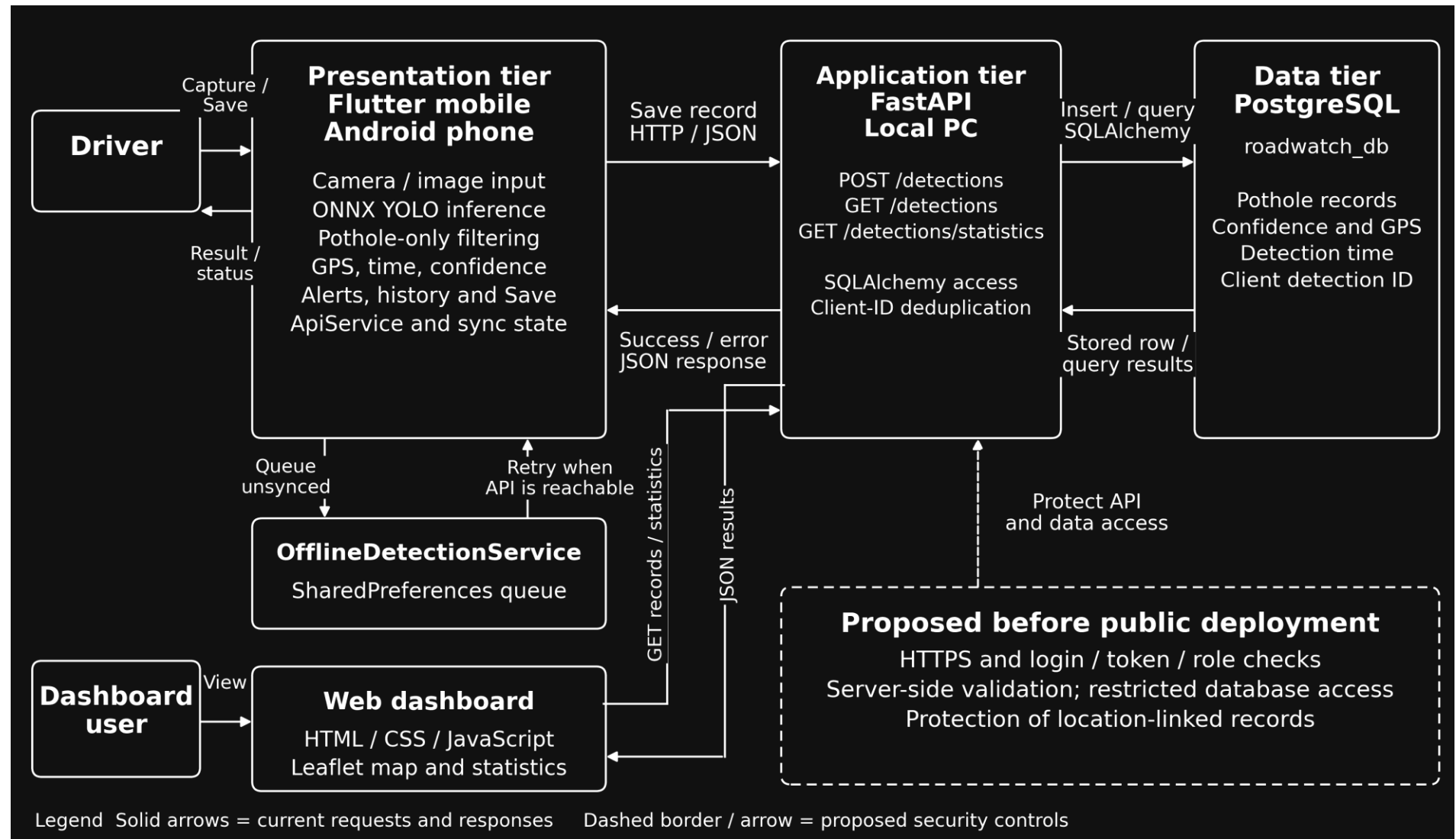
Your architecture must show at least one complete path: Actor → Flutter interface → API/application component → database or external service → response returned to the Flutter interface. Every arrow on this path must have a meaningful label.

The RoadWatch diagram on the next page shows the complete mobile save-and-response path, the local offline queue and the dashboard retrieval path. The model runs in the mobile tier. Dashed elements identify the proposed security controls.

RoadWatch System Architecture

RoadWatch: Pothole and Road Damage Detection Using Mobile Dashcam Imagery with a Web-Based Monitoring System

Three-tier client-server architecture with on-device inference and offline synchronization



Current deployment: Android phone and local FastAPI server on the same Wi-Fi/LAN; API base URL <http://192.168.1.4:8000>. Solid arrows use the current HTTP connection. HTTPS and access controls are proposed for public deployment.

8. Reference Example - Do Not Copy as Your Final Output

The example below illustrates the level of clarity expected in a Flutter-based architecture. It is not a template to copy. Replace it with the actual users, features, services, data, security controls, and deployment choices of your own system.



Figure 1. Template reference example of a request-and-response path in a Flutter-based three-tier architecture.

1. User action	2. HTTPS / JSON request	3. Query or transaction
4. Records or result	5. HTTP status + JSON	6. Updated Flutter interface

9. Recommended Architecture Labels for Flutter Projects

Layer / Area	Possible Label	What It Represents
Presentation	Flutter Web / Flutter App	Screens, widgets, forms, navigation, and user feedback
Client state	Provider / Riverpod / BLoC / controller	Coordinates UI state and user actions
Communication	HTTP client, SDK, or repository/service class	Sends requests and receives responses
Application	REST API, BaaS, serverless function, or custom backend	Routes requests, validates data, and applies business rules
Security	Auth service, token verification, role checks	Protects accounts, data, and restricted operations
Data	PostgreSQL, storage, and important entities	Persists system records and files

For RoadWatch, the Flutter mobile app handles local inference and capture, while the separate browser dashboard displays records from the same FastAPI service.

10. Laboratory Procedure

Phase 1 - Analyze the system (20 minutes)

Review the approved title, problem, users, major features, and available technology choices. Complete the component inventory.

Phase 2 - Choose the architecture (20 minutes)

Select the architecture style and write an initial justification based on security, maintainability, scalability, integration, and deployment.

Phase 3 - Draft the diagram (45 minutes)

Arrange the actors and layers from left to right or top to bottom. Add all major system components before drawing arrows.

Phase 4 - Label data flow (35 minutes)

Label requests and responses using actual workflows, protocols, endpoints or service names, data formats, status/result, and security checks.

Phase 5 - Trace one key transaction (30 minutes)

Use the table in Part D to trace one important user action through the complete architecture.

Phase 6 - Review and improve (15 minutes)

Use the quality checklist. Ask another group or the instructor to identify missing components, unclear arrows, or incorrect layer responsibilities.

Phase 7 - Export and submit (15 minutes)

Export the final diagram, complete the justification, commit the files to GitHub, and submit the required files through MasaoLMS.

11. Part D - Request-to-Response Tracing Exercise

Select one critical workflow from your own system. Examples include login, submitting a report, registering a boarding house, approving a request, uploading an image, requesting a prediction, or retrieving a map-based recommendation.

Selected workflow: Save a pothole detected from an image and synchronize the record with the server.

Expected final result shown to the user: The app shows the saved record and its synchronization status. After upload, the dashboard can display the pothole location and updated statistics.

Step	Actor / Component	Action or Data	Protocol / Mechanism	Expected Result
1	Driver / Flutter camera	Captures or selects a road image and runs detection.	Camera or image input; Flutter action	The selected image is ready for local analysis.
2	ONNX service on the phone	Preprocesses the image, runs the model and accepts Pothole detections.	Local YOLO/ONNX inference	A pothole result and confidence are available.
3	Flutter save logic	The driver taps Save. The app prepares the detection with GPS, time and a client detection ID.	Dart state and device GPS	A record is ready to save and synchronize.
4	Mobile offline queue	If the API is unreachable, retains the pending record locally until a later sync attempt.	OfflineDetectionService; SharedPreferences	The record remains pending without requiring an immediate upload.
5	ApiService	On an online save or later retry, sends the detection fields with the client ID.	HTTP POST /detections; JSON	FastAPI receives the detection request.
6	FastAPI request route	Processes the payload and duplicate identifier. Proposed validation checks class, confidence and coordinate ranges.	REST handler; proposed server validation	An accepted record proceeds to data access; invalid input should receive a clear error.
7	SQLAlchemy / PostgreSQL	Persists the accepted detection or resolves a previously uploaded client detection ID.	SQLAlchemy database operation	A persistent detection record is available in roadwatch_db.
8	FastAPI response	Returns the save result to the requesting phone.	HTTP success/error response; JSON	The phone receives confirmation or an error.
9	Flutter / offline queue	Updates the displayed status. Removes a queued record only after successful synchronization.	Dart state update; local queue update	The user sees the saved/synced result; unsuccessful uploads remain pending.
10	Web dashboard / FastAPI	Requests saved detections and statistics; renders the returned coordinates and values.	GET /detections and GET /detections/statistics; HTTP/JSON	Leaflet markers and statistics reflect server records after refresh.

Live mode uses the same record-saving and synchronization path after automatic capture. The trace above begins with a manual Save action. Authentication and enhanced server-side validation are proposed controls, not confirmed current steps.

12. Part E - Architecture Justification

Write 100–150 words explaining why the selected architecture is appropriate for your system. Address all of the following:

- How the architecture supports the system's main users and features.
- Why the responsibilities are divided among the Flutter client, application/API layer and data layer.
- How the design supports security, maintainability, scalability, integration and deployment.
- Any limitation or risk that the team must manage during implementation.

RoadWatch uses a three-tier architecture to support drivers who detect potholes and dashboard users who review recorded locations. The Flutter app handles camera input, on-device ONNX inference, alerts and temporary offline storage. FastAPI receives detection records and provides data to both clients, while PostgreSQL stores persistent records. This division keeps database access behind the API and allows each layer to be maintained independently. HTTP and JSON connect the components in the current local setup; HTTPS, authentication and authorization are proposed before public deployment. Offline synchronization preserves pending detections when the server is unavailable, and client detection IDs help prevent duplicate uploads. The API and database can be moved to suitable hosting as usage grows. We must still manage false detections, inaccurate GPS readings, phone performance and synchronization failures.

13. Final Quality Checklist

[x] The diagram uses the team's official system title and actual project features.	[x] Deployment locations or platforms are identified.
[x] The selected architecture style is clearly named and justified.	[x] Request and response arrows have clear directions and meaningful labels.
[x] All major actors or user roles are present.	[x] At least one complete request-to-response path is visible.
[x] The Flutter client and important UI/client responsibilities are shown.	[x] Authentication, authorization, validation, and protected resources are represented.
[x] The application/API or backend-as-a-service components are shown.	[x] The diagram has a legend, readable text, consistent symbols, and no crossing arrows that cause confusion.
[x] The database, file storage, and important data domains are shown.	[x] The request-to-response table agrees with the diagram.
[x] External services are included only when the project actually uses them.	[] Every group member can explain an assigned component, trace the complete workflow, and answer questions without reading a script.

14. Submission Requirements

- PDF or PNG of the final architecture diagram in readable landscape orientation.
- Editable diagram source, such as .drawio, .pptx, .fig or the native file of the selected tool.
- Completed request-to-response table and 100-150 word architecture justification.
- Commit the files to the project GitHub repository under a folder such as docs/architecture/.
- Submit the repository link and required files through MasaoLMS using IT110_LA2_Group##_SystemName.
- Deliver a 5-7 minute group presentation followed by 2-3 minutes of questions. Every member must participate.

Academic Integrity and Competency Validation

The output must represent the team's own system and design decisions. AI tools may be used to clarify concepts, but the final architecture, labels, request flow, and justification must be reviewed, understood, and defended by the team. The instructor may ask any member to explain a component or trace a request without assistance.

15. Scoring Rubric - 80 Points

Partial credit may be awarded when evidence is present but incomplete, inconsistent, or technically inaccurate.

For the presentation criterion, points may be reduced for unclear explanation, unequal participation, excessive reading, incorrect tracing, poor time management, or inability to answer questions about the team's own design.

Criterion	Evidence for Full Credit	Points
Architecture correctness and selection	The selected architecture is appropriate for the system requirements, accurately represented, and justified using clear architectural reasoning.	15
System-specific components and layers	Actors, Flutter client, application/API, business logic, data layer, external services, security controls, and deployment are complete and use the team's actual project labels.	12
Request-to-response and data flow accuracy	Arrows, protocols, request/response direction, validation, processing, data access, and returned results form a complete, consistent, and technically coherent workflow.	15
Security and data protection	Authentication, authorization, role checks, input validation, protected resources, data access controls, and appropriate trust boundaries are clearly represented.	8
Technology-stack integration and deployment	The diagram clearly shows how Flutter, the selected backend/API or BaaS, database, storage, external services, and deployment platforms work together.	8
Visual clarity and technical notation	The diagram uses readable text, consistent symbols, meaningful labels, a useful legend, balanced spacing, and directional arrows with minimal visual confusion.	7
Written architecture justification	The 100–150 word explanation clearly connects the selected architecture to the system's users, features, security, maintainability, scalability, integration, deployment, limitations, and risks.	5
Group presentation and oral defense	The presentation is clear, organized, and completed within the assigned time; the final diagram is used effectively; every member participates; the group accurately explains the architecture and request-to-response trace; and questions are answered correctly and confidently.	10

Raw Score	/ 80	Equivalent	_____
Checked by	_____	Date	_____
Remarks			

16. Post-Laboratory Reflection

Which component should contain most of the business rules in your system, and why?

The FastAPI application layer should enforce the shared rules for accepting, storing and retrieving detection records, including validation and duplicate handling. This keeps the rules consistent for the mobile app and dashboard. RoadWatch also needs some rules on the phone: ONNX inference and Pothole filtering run locally so detection can continue without a server connection. The clients can give immediate feedback, while the API should verify records before they enter the central database.

What should the system do when the API, database, or internet connection becomes unavailable?

The app should continue local detection and keep unsynchronized records in the SharedPreferences queue. It should show that the records are pending and retry synchronization when the API becomes reachable. A queued record should be removed only after the server confirms success. If PostgreSQL is unavailable, FastAPI should return a clear error and avoid reporting that the save succeeded. The dashboard should show a connection message rather than misleading empty results. In the current setup, access to the local API depends on LAN connectivity, not necessarily an internet connection.

Which part of the architecture is most security-sensitive?

The FastAPI API and PostgreSQL data layer are the most security-sensitive because they accept and store location-linked records and control central data access. The phone also holds pending location data. Before public deployment, we should add HTTPS, authentication, authorization and server-side validation, restrict database access and protect credentials. These safeguards should cover detection submission and dashboard access, including any future operations that change or remove records.

How would the architecture change if the number of users or stored records increased significantly?

We could move the API and database from the development environment to suitable hosted servers. As demand increases, the API could serve more instances, while PostgreSQL could use indexes, connection pooling and paginated queries. Dashboard requests should load only the records needed for the selected area or time period. The phone would continue running inference locally, which keeps image-processing work on each device. Synchronization would still need duplicate protection when several clients upload pending records at once.

Which component or feature should your team implement first after this activity?

Our next priority should be strengthening the existing save-and-sync path. We should verify that a pothole can be saved while offline, remains available after restarting the app, uploads when the API returns and appears only once in PostgreSQL and the dashboard. This workflow links the main parts of RoadWatch and protects the records collected by drivers. After checking it, we should complete the server-side validation and access controls needed for deployment.